DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS
College of Arts and Sciences | University of the Philippines Visayas
Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020
Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

Building the Runtime Stack

CMSC 124 Activity 15

Prepared by: Rene Andre Bedonia Jocsing

Published: November 12, 2026 **Deadline:** November 15, 2026

Recursion doesn't reuse one set of locals. Every active call needs its own parameters, locals, return point, and links to other frames. Today you'll build that stack at its deepest point, resolve nonlocal identifiers, and then unwind it without dropping a return value.

The programs here are pseudocode. They're written to be read. You don't need another language's syntax rules. We'll use one frame per call, even when a compiler could reuse a frame for a call immediately followed by return. That reuse is called **tail-call optimization**. We won't apply it here. Evaluate expressions from left to right.

You have 60 minutes for the three checkpoints. Keep each worked example beside the checkpoint it supports. Discussion with classmates and the instructor is allowed and encouraged throughout. This activity is worth 22 points. Twelve points reward runtime constructions, and ten reward explanations.

The Frame Contract

Start with this smaller program. In this pseudocode, `main` textually encloses `outer`, and `outer` textually encloses `walk`. A **textual parent** is the procedure whose scope contains another procedure's definition. Use those nesting rules throughout the activity.

```
main:
    base = 2
    result = outer(1)

    outer(n):
        offset = 3

        walk(k):
            local = k
            if k == 0:
                return base + offset + local
            return walk(k - 1)

        return walk(n)
```

An **activation record**, or stack frame, stores the information for one active call. A call **pushes** a fresh frame. Returning **pops** that frame. A **suspended call** is an unfinished caller waiting for a called procedure to return. Use these fields:

Field	Purpose
Parameters and locals	The call's own data
Return address	The instruction to resume after this call returns
Saved state	This call's pending operands, kept while a deeper call runs
Dynamic link	The caller's frame
Static link	The frame for the textually enclosing procedure

The **dynamic chain** follows calls. The **static chain** follows lexical nesting and supports access to nonlocal variables. A recursive call creates a fresh frame even though it runs the same code.

Here, **saved state** records a pending operand in the frame of the call that still needs it. With `return walk(k - 1)`, there's no pending addition, so write `none`. The parameters and locals remain in their own fields. Checkpoint 3 adds a pending operand.

Machines also preserve registers according to their calling conventions, the rules for calls and returns. You don't need to invent registers for this paper model. The drawings use conceptual fields. Their physical layout depends on the implementation.

Worked Example for Checkpoint 1

Suggested time: 28 minutes, including the worked example.

```
main:
    base = 2
    result = outer(1)

    outer(n):
        offset = 3

        walk(k):
            local = k
            if k == 0:
                return base + offset + local
            return walk(k - 1)

        return walk(n)
```

Start at `main` in the smaller program above. Here, nested procedure definitions are available throughout their enclosing scope. A definition describes a procedure's code. Its body runs only when the procedure is called, so reading the definition of `walk` doesn't push a frame.

Trace the calls in execution order. An argument initializes a parameter. `outer(1)` creates `n=1`. Its later `walk(n)` call creates `k=1` in a different frame.

Call that starts	Data initialized in its frame	What it does next
`main`	`base=2`	Calls `outer(1)`. Assignment to `result` waits for its answer.
`outer(1)`	`n=1`, `offset=3`	Calls `walk(n)`, so the next argument is 1.
`walk(1)`	`k=1`, `local=1`	The test `k == 0` is false. Calls `walk(k - 1)` with argument 0.
`walk(0)`	`k=0`, `local=0`	The test is true. Stops just before evaluating the base-case return.

Computing `k - 1` leaves the caller's `k` unchanged. `walk(1)` keeps `k=1` and `local=1` while `walk(0)` uses its own zeros. Every unfinished call is **active**. Only `walk(0)` runs. The other three calls are suspended.

Now fill the return addresses. `return walk(k - 1)` waits for the call to finish. The caller then returns the received value to its own caller.

Frame	Parameters and locals	Return address	Saved state
`main`	`base=2`, `result` pending	program exit	none
`outer(1)`	`n=1`, `offset=3`	`main`: assign received value to `result`	none
`walk(1)`	`k=1`, `local=1`	`outer`: return received value	none
`walk(0)`	`k=0`, `local=0`	`walk(1)`: return received value	none

Each return address in this table identifies an action in that frame's caller. The saved-state entries are all `none` because no expression has a left operand waiting for an addition. The parameters and locals remain in their own fields.

Finally, add the links. Ask two separate questions for each new frame. Who made this call? That frame is the dynamic-link target. Which procedure contains this definition in the source? Its active frame is the static-link target.

For `walk(0)`, the caller is `walk(1)`, but the definition of `walk` sits inside `outer`. The dynamic link reaches `walk(1)`. The static link reaches `outer`. Calling `walk` again executes the same definition inside `outer`, so another recursive frame never becomes its textual parent.

Keep these three destinations in `walk(0)` separate:

Field	Destination	What it identifies
Return address	`walk(1)` resumes its return	A code position
Dynamic link	`walk(1)` frame	The caller's storage
Static link	`outer` frame	The textual parent's storage

Four calls, four frames

Solid arrows are dynamic links to callers. Dashed arrows are static links to textual parents.

Stop inside walk(0)

All four calls are unfinished. Only the top frame runs. The arrows identify the caller and textual parent of each call.

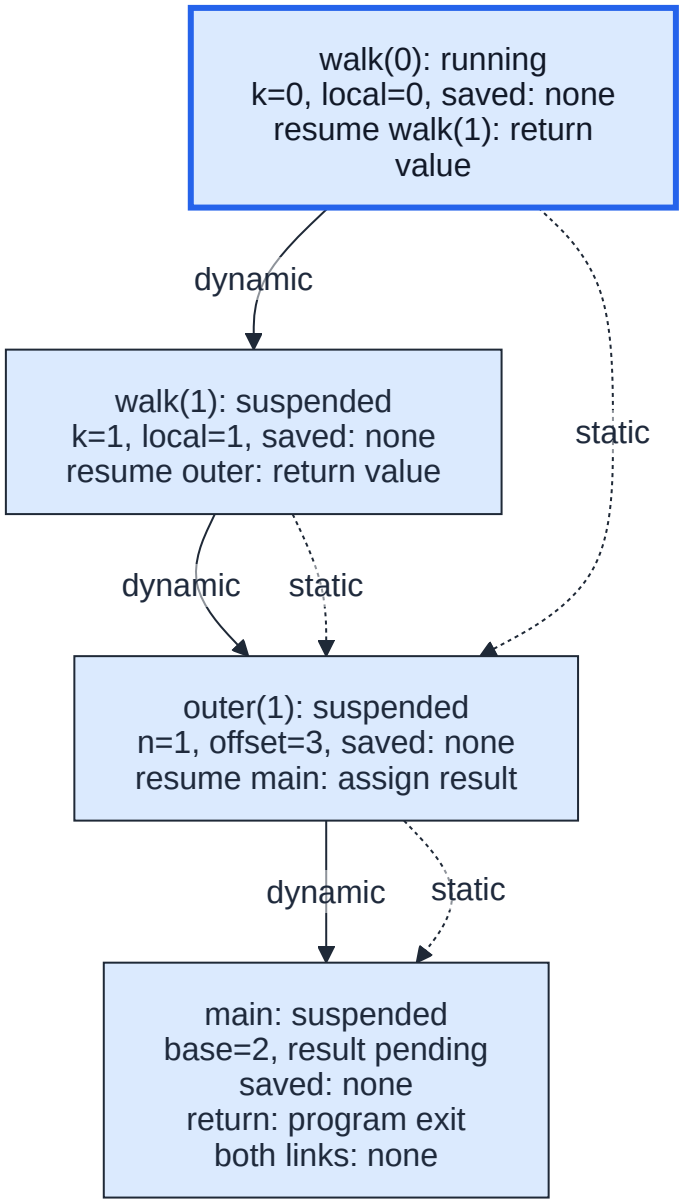


Figure 1: Stop inside walk(0)

In this stack figure, read a solid arrow as “called by” and a dashed arrow as “textually enclosed by.” Both `walk` static links reach the same unfinished `outer(1)` call. `outer` has `main` as both caller and textual parent. `main` is our root frame, so both its links are `none`.

Running Case

All three worked examples use the smaller program's `base=2`, `offset=3`, and `outer(1)`. All three checkpoints start from this larger program. `point P` marks the inspection location. It executes no statement and creates no frame. Stop there before evaluating the following return expression.

```
main:
    base = 10
    result = outer(2)
    print(result)

    outer(n):
        offset = 2

        walk(k):
            local = k
            if k == 0:
                point P
                return base + offset + local
            return walk(k - 1)

        return walk(n)
```

The indentation keeps the same textual nesting. The extra recursive level adds a `walk(2)` frame before `walk(1)` and `walk(0)`. Draw the newest frame at the top. `print(result)` must wait until `outer(2)` returns and `result` receives its value.

Checkpoint 1: Push to Point P (8 Points)

Stop at point P with five unfinished calls. Draw `main`, `outer`, `walk(k=2)`, `walk(k=1)`, and `walk(k=0)`. Use the five fields from the frame contract.

1. (Construction: 5 points) Start with `main`. Fill each frame's parameters, initialized locals, saved state, and both links. For each return address, identify the caller and the action it performs with the returned value. Mark `result` as pending. Record any pending operand in its owning frame, or write `none`.

2. (Explanation: 3 points) Answer all three, one point each.

a. Why does each of the three unfinished `walk` calls need a frame of its own?
b. Why do their dynamic links follow the call sequence?
c. Why do all three static links reach `outer`, even when another `walk` call is the caller?

Resolve the Nonlocal Names

A lookup finds the declaration used by an identifier and reads its value. **Nonlocal** identifiers have declarations outside the current procedure. Here, `local` belongs to `walk`, `offset` belongs to `outer`, and `base` belongs to `main`.

Our programs use static scope. Start in the current frame. If that procedure has no matching declaration, follow its static link to the enclosing procedure's frame. Repeat until the declaration is found. Stop at that declaration. The source nesting determines this path.

Worked Example for Checkpoint 2

Suggested time: 19 minutes, including the worked example.

Return to the smaller program's `walk(0)` frame. It contains parameter `k=0` and local `local=0`. Resolve each identifier in `base + offset + local` separately, starting from that frame each time.

Identifier	Frames examined	Why lookup stops	Value
`local`	`walk(0)`	`walk` declares `local`	0
`offset`	`walk(0) -> outer`	`walk` has no `offset`. `outer` declares it.	3
`base`	`walk(0) -> outer -> main`	Neither `walk` nor `outer` declares `base`. `main` does.	2

Substitute those values into the expression. `walk(0)` returns `2 + 3 + 0 = 5`. The original recursive return has no pending addition, so the remaining calls pass that 5 outward:

Frame returning	Value	Frame popped	Caller resumes by
`walk(0)`	5	`walk(0)`	`walk(1)` returns the received 5
`walk(1)`	5	`walk(1)`	`outer` returns the received 5
`outer`	5	`outer`	`main` assigns `result=5`

The return address in each departing frame supplies the caller's resume action. The dynamic link reaches the caller's frame. Its parameters and locals still exist because that caller hasn't returned yet. `main` continues after the assignment when `outer` finishes.

A dynamic-link search for `offset` would visit `walk(1)` before `outer`. It happens to find the same value in this program because the recursive caller declares no `offset`. That agreement doesn't prove the lookup rule correct. Static scope selects declarations through the source's enclosing scopes.

Checkpoint 2: Walk the Correct Chain (8 Points)

Use the larger program's stack at point P. Its values differ from the worked example:

Frame	Parameters and locals	Dynamic link	Static link
`walk(k=0)`	`k=0, local=0`	`walk(k=1)`	`outer`
`walk(k=1)`	`k=1, local=1`	`walk(k=2)`	`outer`
`walk(k=2)`	`k=2, local=2`	`outer`	`outer`
`outer`	`n=2, offset=2`	`main`	`main`
`main`	`base=10, result` pending	none	none

1. (Construction: 4 points) For `local`, `offset`, and `base`, write every frame examined, the declaration selected, and the value read. Start each lookup at `walk(0)`. Compute the base-case return expression.

2. (Explanation: 4 points) Explain why static scope uses static links (1 point). Explain how the caller and textual parent differ here (1 point). Distinguish the return address from both links (1 point). State whether `walk(1)` has a pending operand when `walk(0)` returns in this original program (1 point).

Transfer Through the Unwind

Worked Example for Checkpoint 3

Suggested time: 13 minutes, including the worked example.

In the smaller program above, `walk(1)` reads its own `k=1`. It keeps operand 1 in its saved-state field and calls `walk(0)`. That new frame receives `k=0`, sets `local=0`, and takes the base case. Its return expression is still `2 + 3 + 0 = 5`.

The following figure shows value transfer for the modified program in Checkpoint 3. Its arrows carry operands and results, so they aren't the frame links from the earlier stack figure.

Keep the operand in its caller

Resume the modified walk(1)

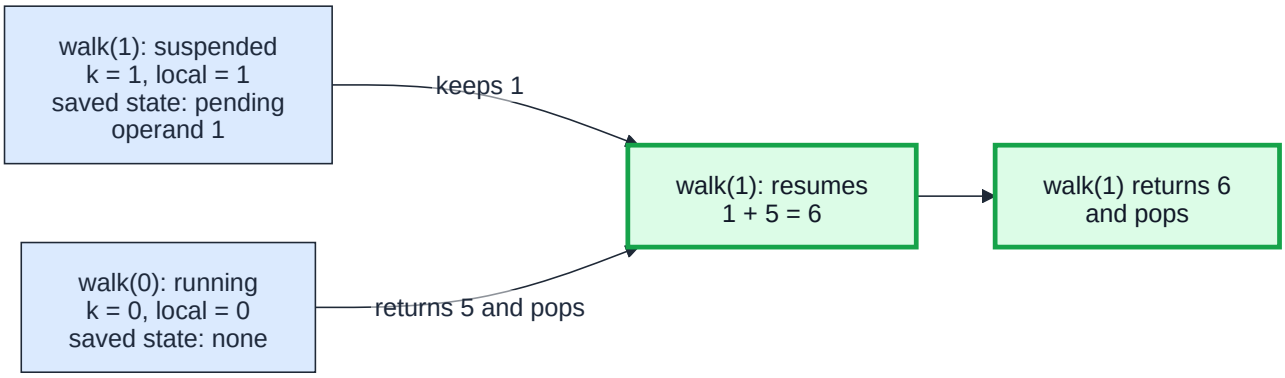


Figure 2: Resume the modified walk(1)

When `walk(0)` returns, use its return address to resume `walk(1)` at the pending addition. Combine the returned 5 with the saved 1. Only then can `walk(1)` return and its frame pop.

Frame returning	Value	Frame popped	Caller resumes by
`walk(0)`	5	`walk(0)`	`walk(1)` computes `1 + 5 = 6`
`walk(1)`	6	`walk(1)`	`outer` returns the received 6
`outer`	6	`outer`	`main` assigns `result=6`

The saved 1 belongs to `walk(1)`. Returning from `walk(0)` removes the zero frame, so it doesn't discard that operand. The original program returned 5. The modified program returns 6 because `walk(1)` now performs its pending addition.

Checkpoint 3: Carry the Value Home (6 Points)

Rerun each program from `main`, changing only the recursive return inside `walk`, with this modified program:

```
main:
    base = 10
    result = outer(2)
    print(result)

    outer(n):
        offset = 2

        walk(k):
            local = k
            if k == 0:
                point P
                return base + offset + local
            return k + walk(k - 1)

        return walk(n)
```

Keep the base-case return unchanged. The calls, parameter values, and links stay the same. What changes is the work waiting in each suspended recursive frame. Left-to-right evaluation reads that frame's `k` before making the deeper call. The addition must wait for the deeper call's result.

Stop at point P. Each suspended recursive frame now keeps its own `k` as a pending operand. `outer` still waits to return the final `walk` value, and `main` still waits to assign it to `result`.

1. (Construction: 3 points) Start by evaluating the base-case expression. Trace each returned value, frame pop, caller's resume action, and pending addition until `main` receives `result`. Attach each addition to the frame that kept its operand.

2. (Explanation: 3 points) Answer all three, one point each.

a. Why do the two suspended calls retain different `k` values?
b. What would one shared frame overwrite before a caller could resume?
c. Which frame keeps each pending operand and uses it after the deeper call returns?

If the Language Chose Dynamic Scope Instead

The three checkpoints all use static scope. This section compares two ways to implement dynamic scope, the alternative rule from Activity 9. Under **dynamic scope**, lookup checks the current call and then its callers. It selects the first matching active declaration.

Deep access performs that search through dynamic links. For `offset` at point P, the larger program's path would be `walk(0) -> walk(1) -> walk(2) -> outer`. The first three frames have no `offset`, so lookup reads `outer`'s 2 and stops. It never visits `main`. If no matching declaration exists anywhere on the path, lookup fails.

One form of **shallow access** keeps a separate stack of active **bindings**, associations between identifiers and their storage. When a declaration becomes active, push its binding. When its scope ends, pop that binding. A read uses the top binding directly. For the same point P:

Identifier	Binding values, oldest to newest	Value read now	Value after `walk(0)` returns
`local`	2, 1, 0	0	1
`offset`	2	2	2
`base`	10	10	10

The three `local` bindings belong to different calls. Removing the newest exposes `walk(1)`'s 1. It doesn't alter either remaining value. If a binding stack is empty, a lookup of that identifier fails.

Deep access examines frames on each read. Shallow access avoids that traversal but must maintain binding stacks at scope entry and exit. Both implement dynamic scope. Static scope instead uses the declarations selected from source nesting and reaches their active frames through static links.

Where a Block Lives

A **block** groups statements and can introduce a scope for its declarations. As a separate storage example, replace the larger program's `walk` body with this version:

```
walk(k):
    local = k
    if k == 0:
        temp = local * 2
        return base + offset + temp
    return walk(k - 1)
```

Here, the `if` body is a block, and `temp` belongs to that block alone. `walk(1)` skips the body, so it creates no active `temp` binding. `walk(0)` enters it, initializes `temp = 0 * 2 = 0`, and uses that value in its return. Entering the block makes no procedure call.

One implementation gives each block a separate record. Another reserves storage inside the enclosing procedure's frame. Use the second model here. Each `walk` frame can reserve a slot for `temp`, even while that block's binding is inactive. Reserving space doesn't run the initialization or make `temp` visible outside its block.

With these fixed-size variables, the compiler can determine the slot's **offset**, its distance from the frame's base. That storage distance is independent of the program variable `offset` in `outer`. No extra frame is needed for this block in our model.

The procedure frame must hold all block variables whose lifetimes overlap. An inner block can be active while its enclosing block remains active, so both need storage. Blocks whose variable lifetimes never overlap may reuse slots.

Storage reuse and shadowing answer different questions. **Shadowing** occurs when an inner declaration hides an outer declaration of the same identifier. The outer binding can remain alive while hidden. It still needs storage for a value that becomes visible again when the inner scope ends.

Where That Leaves You

Your three traces give the runtime three jobs. A call creates storage for its own parameters and locals. A lookup reaches the declaration selected by the language's scope rule. A return delivers a value and resumes a caller that still has work to finish.

The Prolog problem set, **ps4**, adds saved search information. A **choice point** records remaining alternatives and enough state to try them later. **Backtracking** returns to the most recent remaining

choice point, undoes bindings made since it, and tries the next alternative. This requires information beyond the ordinary call-and-return frames you drew. It needn't occupy the same physical record as a call frame.

That problem set asks you to distinguish calls and returns from a search trying another clause. Your interpreter also needs to preserve each unfinished call's data and resume action. The drawings show what that bookkeeping must accomplish.

Phew, now we're done with all fifteen activities. Congrats!